

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
Кафедра «Информационная безопасность в вычислительных системах и сетях»

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к практическому занятию № 2

«Доказательство корректности и экспериментальная оценка трудоёмкости»

по дисциплине «Алгоритмы и структуры данных»

Параметр	Значение
Направление подготовки	03.03.01 Прикладные математика и физика
Направленность	03.03.01 Прикладные физика и математика
Семестр	7
Продолжительность	2 академических часа
Компетенция / индикатор	ПК-4 / ПК-4.4
Статус	Проект методических указаний

Ростов-на-Дону
7 сентября 2026 г.

1. Паспорт практического занятия

Основание разработки. Актуализированная РПД дисциплины «Алгоритмы и структуры данных», раздел 1 «Основы алгоритмизации и анализа сложности»: корректность и инварианты, O -, Ω - и Θ -оценки, лучший, средний и худший случаи, экспериментальная проверка трудоёмкости.

Преимственность. Практическая работа № 1 ввела контракт $\{P\} A \{Q\}$, трассировку и базовый инвариант. Во второй работе студент доказывает полную корректность циклического алгоритма и проверяет теоретическую модель серий воспроизводимых измерений времени и памяти.

Цель. Научиться связывать доказательство корректности и асимптотическую оценку с корректно поставленным вычислительным экспериментом, отделяя свойства алгоритма от особенностей конкретного запуска.

1.1. Планируемые результаты

По завершении работы обучающийся:

- формулирует инвариант цикла и функцию-вариант, доказывая частичную корректность и завершение;
- получает теоретические оценки времени, рабочей и результирующей памяти;
- планирует серию измерений с контролем размера входа, прогрева, повторов и внешних факторов;
- измеряет медианное время и пик выделенной памяти средствами Python;
- сопоставляет наблюдаемый порядок роста с теоретическим и объясняет расхождения;
- оформляет воспроизводимый протокол с кодом, исходными данными и результатами.

1.2. Прослеживаемость

Элемент	Деятельность	Проверяемый артефакт
ПК-4 / ПК-4.4	Применение средств тестирования и профилирования	Код эксперимента и таблица измерений
Базовый уровень	Определение операций и оценка простого цикла	Теоретическая модель $T(n)$, $M(n)$
Средний уровень	Профилирование и интерпретация расхождений	График/таблица, коэффициенты роста, вывод
Текущий контроль	Практическая работа № 2	Отчёт, .py, .csv, ответы при защите

2. Подготовка и ресурсы

Необходимо знать материал практической работы № 1, уметь работать с циклами, списками и функциями Python, понимать обозначения O , Ω и Θ . Перед занятием повторите понятия предусловия, постусловия и инварианта.

Программная среда:

- Python 3.12 или новее; стандартные модули time, statistics, tracemalloc, gc, platform и csv;
- JupyterLab, VSCodium, Visual Studio Code либо командная строка;
- не менее 200 МБ свободной оперативной памяти для рекомендуемой серии;
- измерения времени и памяти выполняются отдельно, поскольку tracemalloc замедляет программу.

Ограничения. Не запускайте нагрузочный эксперимент на общем учебном компьютере с размерами, вызывающими подкачку или зависание. Начинайте с малых n , наблюдайте потребление памяти и прекращайте серию при заметном ухудшении отзывчивости системы.

3. Краткие теоретические сведения**3.1. Частичная и полная корректность**

Частичная корректность означает: если алгоритм завершился при выполненном предусловии P , то истинно постусловие Q . Полная корректность дополнительно требует доказать завершение. Для цикла обычно используют инвариант I и целочисленную неотрицательную функцию-вариант V , которая строго уменьшается на каждой итерации.

Шаг	Что доказывается	Контрольная формула
Инициализация	Инвариант истинный до первого шага	$P \Rightarrow I$
Сохранение	Тело цикла сохраняет инвариант	$I \wedge B \Rightarrow I$ после тела
Завершение	Инвариант и выход дают результат	$I \wedge \neg B \Rightarrow Q$
Терминация	Вариант неотрицателен и уменьшается	$V \geq 0; V_{\text{new}} < V_{\text{old}}$

3.2. Теоретическая модель ресурсов

Асимптотическая оценка описывает тенденцию при росте n , а не время в секундах. Для линейного алгоритма ожидается $T(n) \approx an + b$. Если n удваивается, отношение $T(2n)/T(n)$ стремится к 2 после уменьшения влияния постоянных накладных расходов.

Показатель экспериментального порядка роста для двух точек вычисляют как $p = \log(t_2/t_1) / \log(n_2/n_1)$. При удвоении n формула упрощается до $p = \log_2(t_2/t_1)$. Для $\theta(n)$ ожидается $p \approx 1$, для $\theta(n^2)$ — $p \approx 2$.

Память следует описывать двумя величинами: рабочая память без результата и общий объём дополнительных выделений, включая результат. Например, формирование массива префиксных сумм требует $\theta(1)$ рабочих скаляров, но $\theta(n)$ памяти для возвращаемого массива.

3.3. Правила измерительного эксперимента

- формировать входные данные до запуска таймера;
- использовать детерминированный генератор или фиксированную формулу;

- выполнить прогревочный запуск;
- проводить не менее 7 повторов и использовать медиану, устойчивую к единичным выбросам;
- не включать печать, чтение файлов и построение графика в измеряемый фрагмент;
- измерять память отдельно; tracemalloc учитывает выделения Python, но не всю резидентную память процесса;
- фиксировать версию Python, архитектуру, операционную систему (ОС), размеры входа и число повторов;
- не отбрасывать неудобные результаты: выбросы пометать и объяснять по заранее заданному правилу.

4. Порядок выполнения

1. Запишите контракт алгоритма и уточните, что считается размером входа n .
2. Сформулируйте инвариант, докажите инициализацию, сохранение и получение постусловия.
3. Задайте функцию-вариант и докажите завершение.
4. Получите теоретические $T(n)$, рабочую память и память результата.
5. Составьте план эксперимента: размеры n , способ генерации данных, прогрев, число повторов и агрегирование.
6. Реализуйте алгоритм и тесты корректности до измерения производительности.
7. Выполните временную серию; сохраните сырые повторы и медианы.
8. Отдельно измерьте пик выделенной памяти для каждого n .
9. Вычислите отношения при удвоении n и экспериментальный показатель p .
10. Сопоставьте теорию и эксперимент; перечислите ограничения измерений.

5. Демо-пример: массив префиксных сумм

5.1. Постановка и контракт

Для последовательности конечных вещественных измерений $x[0 \dots n-1]$ построить массив y той же длины, где $y[i]$ равен сумме элементов $x[0] + \dots + x[i]$. Пустой вход допустим и даёт пустой результат.

Предусловие P: $n \geq 0$; длина x равна n ; все $x[i]$ — конечные вещественные числа.

Постусловие Q: длина y равна n и для каждого $0 \leq i < n$ выполнено $y[i] = \sum_{j=0 \dots i} x[j]$.

5.2. Псевдокод и доказательство

```
PREFIX_SUMS(x)
  y ← array of n zeros
  s ← 0
  for i ← 0 to n - 1 do
    s ← s + x[i]
    y[i] ← s
  return y
```

Инвариант. Перед итерацией с индексом i значение s равно сумме $x[0 \dots i - 1]$, а для каждого $j < i$ элемент $y[j]$ равен сумме $x[0 \dots j]$.

Инициализация. При $i = 0$ сумма пустого префикса равна 0, обработанных элементов у ещё нет.

Сохранение. После $s \leftarrow s + x[i]$ переменная s становится суммой $x[0 \dots i]$; присваивание $y[i] \leftarrow s$ устанавливает нужный префикс, поэтому инвариант верен для $i + 1$.

Завершение. После n итераций условие $j < n$ охватывает весь y , и инвариант совпадает с Q .

Терминация. Функция-вариант $V = n - i$ перед телом цикла неотрицательна и уменьшается на 1; следовательно, цикл завершится.

Теоретическая оценка. Выполняется n итераций: $T(n) = \theta(n)$. Рабочая память – $\theta(1)$, память возвращаемого массива – $\theta(n)$, общий дополнительный объём – $\theta(n)$.

5.3. Проверка функциональной корректности

```
from math import isclose

def prefix_sums(x: list[float]) -> list[float]:
    out = [0.0] * len(x)
    s = 0.0
    for i, value in enumerate(x):
        s += value
        out[i] = s
    return out

assert prefix_sums([]) == []
assert prefix_sums([2.5]) == [2.5]
assert prefix_sums([1.5, -0.5, 2.0, -3.0]) == [1.5, 1.0, 3.0, 0.0]
assert prefix_sums([0.0, 0.0]) == [0.0, 0.0]
assert all(isclose(a, b) for a, b in zip(
    prefix_sums([0.1, 0.2, 0.3]), [0.1, 0.3, 0.6]))
```

5.4. Код измерительного эксперимента

```
from time import perf_counter_ns
from statistics import median
import gc, platform, sys, tracemalloc

def make_data(n: int) -> list[float]:
    return [((i * 37) % 101 - 50) / 10 for i in range(n)]

def median_time_ms(x: list[float], repeats: int = 9) -> float:
    prefix_sums(x) # nporpev
    samples = []
    for _ in range(repeats):
        gc.disable()
        try:
            t0 = perf_counter_ns()
            prefix_sums(x)
            samples.append((perf_counter_ns() - t0) / 1_000_000)
        finally:
            gc.enable()
    return median(samples)

def peak_kib(x: list[float]) -> float:
    tracemalloc.start()
```

```

prefix_sums(x)
_, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
return peak / 1024

print(sys.version.split()[0], platform.machine())
for n in (10_000, 20_000, 40_000, 80_000, 160_000, 320_000):
    x = make_data(n)
    print(n, median_time_ms(x), peak_kib(x))

```

Если на конкретном компьютере отдельный запуск длится менее 1 мс, увеличьте размеры n или число внутренних запусков. Нельзя округлять время до целых миллисекунд до завершения анализа.

5.5. Иллюстративные результаты

Ниже приведена контрольная серия, полученная 7 сентября 2026 г. на Python 3.12.13, архитектура x86_64, 9 повторов. КиБ — кибибайт, равный 1024 байтам. Эти числа не являются нормативом производительности: студент формирует собственную таблицу.

n	Медиана, мс	Пик, КиБ	$t(n)/t(n/2)$	$M(n)/M(n/2)$
10 000	0,437	310,3	—	—
20 000	0,937	622,8	2,14	2,01
40 000	2,031	1 247,8	2,17	2,00
80 000	4,217	2 497,8	2,08	2,00
160 000	7,830	4 997,8	1,86	2,00
320 000	15,786	9 997,8	2,02	2,00

Интерпретация. При удвоении n время в основном возрастает примерно в 2 раза, а память — почти точно в 2 раза. Значения p для соседних временных точек находятся приблизительно в диапазоне 0,90–1,12. Это согласуется с $\Theta(n)$; отклонения объясняются накладными расходами, кэшированием и планированием процессов.

Недопустимый вывод: «эксперимент доказал $\Theta(n)$ ». Корректная формулировка: «результаты не противоречат теоретической линейной модели на исследованном диапазоне n ».

6. Индивидуальные домашние задания

Для своего варианта студент выполняет полный цикл: контракт, доказательство полной корректности, теоретические $T(n)$ и $M(n)$, не менее пяти тестов, временную и память-серию минимум для шести размеров, расчёт отношений и p , интерпретацию и ограничения.

№	Вариант	Условие
1	Префиксные минимумы	Для каждого i вернуть минимум $x[0 \dots i]$; при равенстве хранить первое значение.
2	Префиксное число превышений	Для каждого i вернуть количество элементов $x[0 \dots i]$, превышающих порог τ .

№	Вариант	Условие
3	Накопленная энергия	Построить $e[i] = \sum_{j=0 \dots i} x[j]^2$ для ряда измерений.
4	Последовательные приращения	Построить $d[i] = x[i+1] - x[i]$ для $n \geq 1$.
5	Нормирование по максимуму	Разделить элементы на $\max x[i] $; отдельно определить поведение нулевого массива.
6	Префиксные средние	Построить среднее каждого непустого префикса за один проход.
7	Суффиксные суммы	Для каждого i вычислить сумму $x[i \dots n-1]$, проходя массив справа налево.
8	Накопленный интеграл	По равномерной сетке с шагом h построить накопленную сумму трапеций.
9	Префиксные смены знака	Для каждого i вернуть число строгих смен знака в $x[0 \dots i]$; нули пропускать.
10	Префиксный максимум с индексом	Для каждого i вернуть значение максимума и индекс его первого появления.
11	Контрольная сумма	Для массива целых чисел построить $c[i] = (c[i-1] + x[i]) \bmod m$, $m > 0$.
12	Суммы сегментов	Накапливать сумму, начиная новый сегмент после специального маркера; вернуть сумму для каждой позиции.

7. Требования к эксперименту и отчёту

Обязательный комплект сдачи: PR02_Фамилия_ВариантNN.pdf или .docx, исходный файл .py и таблица в формате CSV (.csv) с сырыми результатами. В отчёте должны быть:

- контракт и определение размера входа n ;
- инвариант, доказательство трёх условий и функция-вариант;
- теоретические оценки времени, рабочей памяти и памяти результата;
- описание аппаратно-программной среды и протокол измерений;
- не менее пяти функциональных тестов, включая границы;
- не менее шести размеров n и не менее семи временных повторов для каждого размера;
- таблица сырых времён, медиан, пиков памяти, отношений роста и p ;
- вывод о согласованности с теорией и не менее трёх ограничений эксперимента.

Воспроизводимость и академическая добросовестность. Укажите версию Python, ОС/архитектуру, команду запуска и все изменённые параметры. Раскройте использование внешнего кода и генеративного ИИ, перечислите самостоятельно проверенные элементы. Не передавайте внешним сервисам персональные или закрытые данные.

8. Критерии оценивания

Компонент	Баллы	Критерий полного выполнения
Контракт и тесты	10	P и Q однозначны; минимум пять тестов до профилирования
Инвариант	20	Доказаны инициализация, сохранение и получение Q
Завершение	10	Корректная функция-вариант и доказательство её убывания
Теоретический анализ	10	Обоснованы время, рабочая память и память результата

Компонент	Баллы	Критерий полного выполнения
План эксперимента	15	Размеры, прогрев, повторы, медиана и контролируемые условия
Результаты и анализ	25	Сырые данные, медианы, память, отношения, p и содержательная интерпретация
Отчёт и воспроизводимость	10	Среда и команда запуска указаны; файлы достаточны для повтора

Шкала: 91–100 баллов – «отлично»; 76–90 – «хорошо»; 61–75 – «удовлетворительно»; менее 61 – доработка. Без функциональных тестов или при измерении заведомо неверной реализации экспериментальная часть не засчитывается.

9. Вопросы для самопроверки

1. Чем частичная корректность отличается от полной?
2. Какие четыре части содержит доказательство полного результата для цикла?
3. Почему функция-вариант должна быть ограничена снизу?
4. Как инвариант для префиксных сумм связан с постусловием?
5. Почему асимптотическая оценка не предсказывает точное время в секундах?
6. Какое отношение $T(2n)/T(n)$ ожидается для линейного и квадратичного алгоритмов?
7. Что показывает экспериментальный показатель p ?
8. Почему входные данные формируют вне измеряемого участка?
9. Для чего нужен прогревочный запуск?
10. Почему медиана обычно предпочтительнее среднего для коротких измерений?
11. Почему время и память измеряются отдельно?
12. Что именно измеряет tracemalloc и чего он не измеряет?
13. Чем рабочая память отличается от памяти результата?
14. Почему единичная точка не позволяет оценить порядок роста?
15. Как корректно описать экспериментальный выброс?
16. Почему нельзя утверждать, что эксперимент сам по себе доказал $\Theta(n)$?
17. Какие сведения необходимы для повторения серии на другом компьютере?
18. Какие действия требуются, если измерение занимает меньше разрешения таймера?

10. Методические рекомендации преподавателю

Этап	Время	Деятельность
Актуализация ПР-01	10 мин	Проверить контракт, инвариант и обозначения сложности
Полная корректность	20 мин	Разобрать функцию-вариант и отличие частичной корректности
Демо-эксперимент	25 мин	Запустить серию и обсудить медиану, отношения и p
Индивидуальная работа	25 мин	Составить доказательство и план серии своего варианта
Интерпретация	5 мин	Сопоставить несколько результатов разных компьютеров
Выдача задания	5 мин	Зафиксировать файлы, критерии и ограничения

При защите рекомендуется предложить студенту объяснить один выброс, изменить диапазон n или указать, как изменится память при потоковой реализации без хранения всего результата.

11. Основные источники

1. Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 4th ed. MIT Press, 2022.
2. Skiena S. S. The Algorithm Design Manual. 3rd ed. Springer, 2020.
3. Morin P. Open Data Structures. Open textbook. Актуальная электронная редакция.
4. Python Software Foundation. time — Time access and conversions; tracemalloc — Trace memory allocations. Python 3 Documentation. <https://docs.python.org/3/> (дата обращения: 07.09.2026).

12. Лист самоконтроля перед сдачей

- ☐ Алгоритм прошёл функциональные тесты до начала измерений.
- ☐ Инвариант согласован с границами цикла, функция-вариант убывает.
- ☐ Теоретические оценки разделяют рабочую память и память результата.
- ☐ Вход создаётся вне таймера, выполнен прогрев, сохранены сырые повторы.
- ☐ Временная серия содержит не менее шести n и семи повторов.
- ☐ Память измерена отдельно от времени.
- ☐ Рассчитаны отношения при росте n и показатель p .
- ☐ Вывод отделяет теоретическое доказательство от экспериментального подтверждения.
- ☐ Указаны версия Python, платформа, команда запуска и ограничения.
- ☐ Раскрыто использование внешнего кода и генеративного ИИ.